https://github.com/tejasmali46/AIML_lab

Assignment 2
Prolog

Name	Tejas Vinod Mali
PRN	202401040083
Batch	A1
Division	A
Roll No.	25
Subject	Artificial Intelligence and Machine Learning

Title

Design and implementation of a Knowledge-Based System in Prolog to represent facts and relationships among superheroes and villains (the Avengers universe), and to derive new relationships such as allies, rivals, and life status using logical rules.

Aim

To study the fundamentals of logic programming using Prolog and to implement a knowledge base consisting of facts and rules that models relationships between heroes, their powers, weapons, and teams, and to derive new facts through rule-based inference and backtracking.

Theory

Prolog (Programming in Logic) is a declarative programming language widely used in **AI and expert systems**. Instead of specifying step-by-step instructions, Prolog uses **facts and rules** to represent knowledge.

A Prolog knowledge base contains:

- **Facts:** Statements that are true, such as member(ironman, avengers).
- **Rules:** Relationships defined using Head :- Body, where the head is true when the body conditions are satisfied.

When a **query** is given, Prolog uses **unification** to match it with facts and rules. If a solution fails, it uses **backtracking** to try other possible solutions.

Observation

During execution, it was observed that facts of the same predicate (such as member/2) are tried in the order they appear in the knowledge base, and Prolog returns solutions one at a time, prompting for backtracking (';') to obtain further alternatives. Rules built using conjunction (',') require every sub-goal to succeed, while the ally/2 and fight/2 rules demonstrate how new relationships can be derived purely from existing facts without being stated explicitly. It was also observed that the \== operator was necessary in the ally rule to prevent a hero from being listed as their own ally.

Complete Code

```
%This relation is between hero and team

member(ironman,avengers).
member(captain,avengers).
member(thor,avengers).
member(loki,villains).
member(thanos,villains).

%.....

%This relation is between hero and power

power(ironman,technology).
power(captain,strength).
power(thor,thunder).
power(thanos,strength).

%.....

%This relation is between hero and weapon

weapon(ironman,armor).
weapon(captain,shield).
weapon(thor,hammer).
weapon(thanos,gauntlet).

%.....

%This is relation between hero and age

age(ironman,45).
age(captain,39).
age(thor,1500).
age(thanos,1000).

% .....

%This is relation between age and status
```

```
status(young,45).
status(young,39).
status(old,1500).
status(old,1000).

%.....................................................................

%This relation is between team and enemy team

enemy(avengers,villains).

%.....................................................................

%This relation is ally between two heroes of same team

ally(X,Y):-
        member(X,T),
        member(Y,T),
        X \== Y.

%.....................................................................

%This relation is fight between hero and enemy hero

fight(X,Y):-
        member(X,T1),
        member(Y,T2),
        enemy(T1,T2).

%.....................................................................

%This is relation between hero and their life status

lifestatus(Y,A,S):-
        age(Y,A),
        status(S,A).


/* ally(ironman,Y).
Y = captain ;
Y = thor.

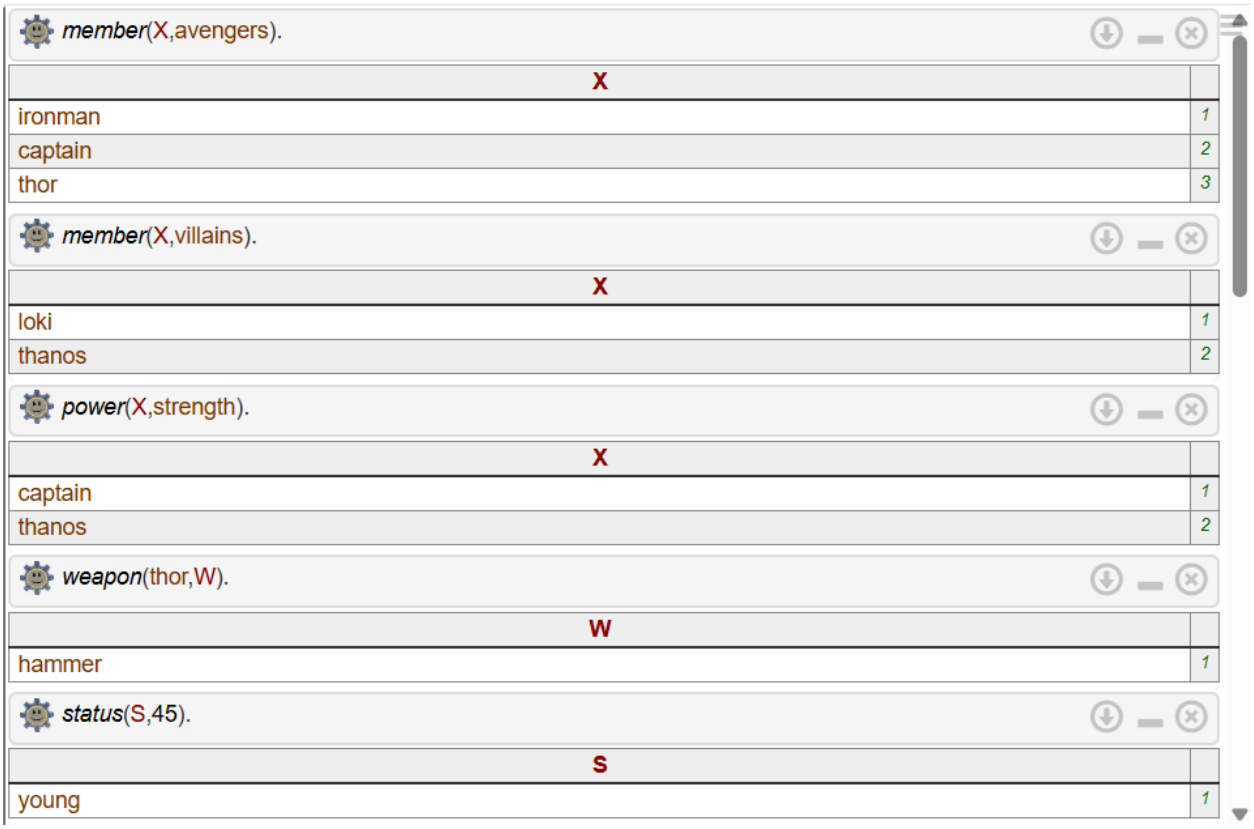
fight(X,Y).
X = ironman,
Y = loki ;
X = ironman,
Y = thanos ;
X = captain,
Y = loki ;
X = captain,
Y = thanos ;
X = thor,
Y = loki ;
X = thor,
Y = thanos.
```

lifestatus(thor,A,S).
A = 1500,
S = old.

lifestatus(captain,A,S).
A = 39,
S = young.

*/

Output :



member(X,avengers).

X	
ironman	1
captain	2
thor	3

member(X,villains).

X	
loki	1
thanos	2

power(X,strength).

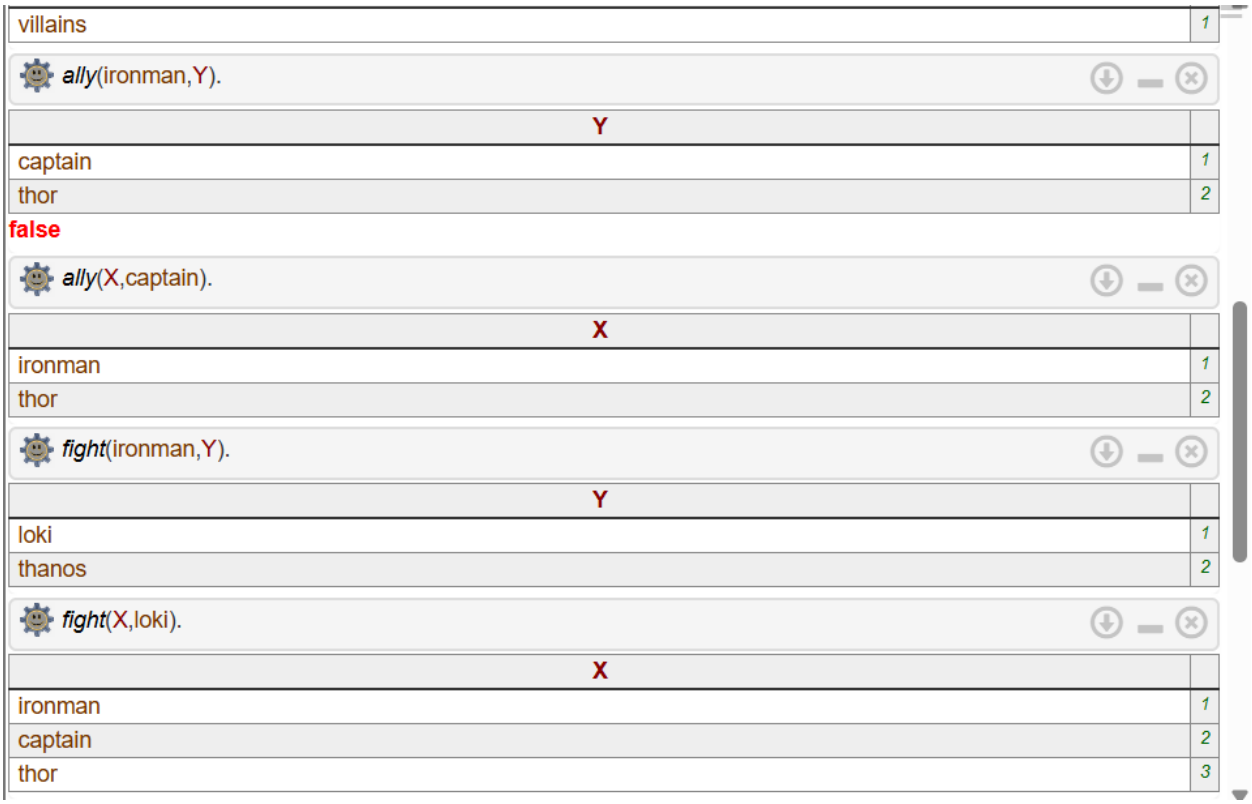
X	
captain	1
thanos	2

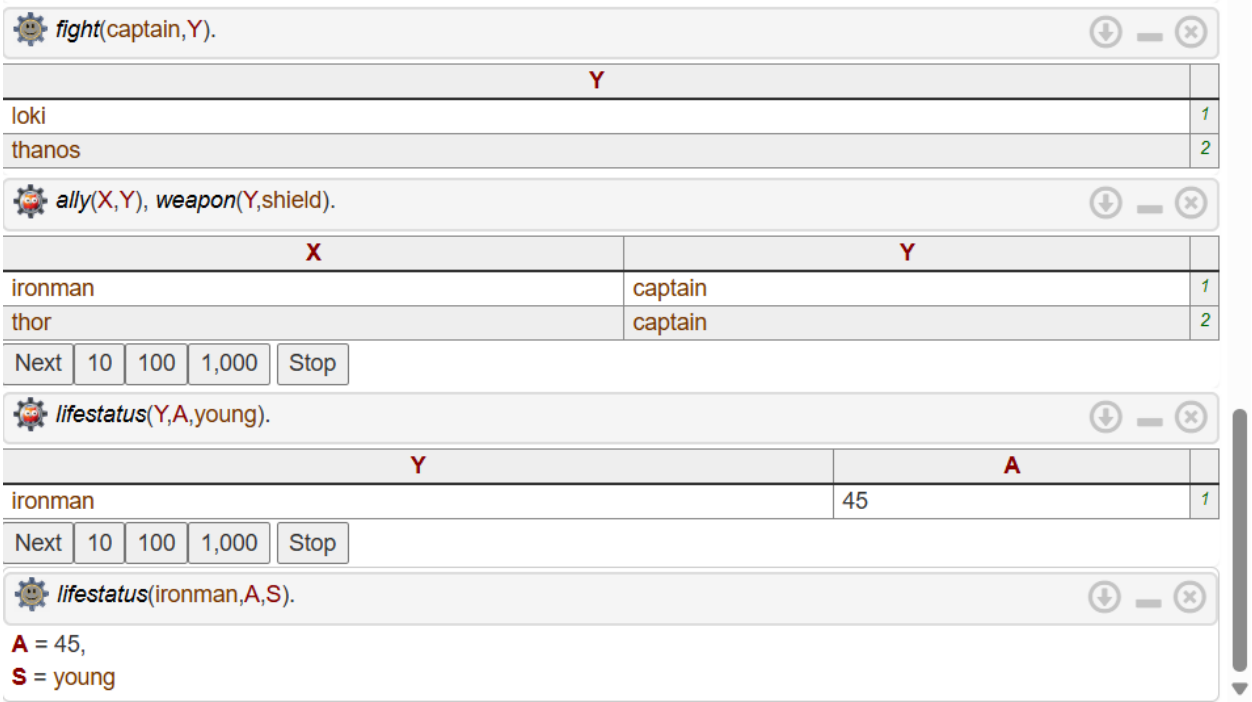
weapon(thor,W).

W	
hammer	1

status(S,45).

S	
young	1





Result Analysis

The queries executed against the knowledge base confirmed that the rules correctly derive relationships consistent with the underlying facts. The ally/2 rule correctly grouped heroes

belonging to the same team while excluding self-pairing. The fight/2 rule correctly identified cross-team combinations between the avengers and villains teams based on the enemy/2 fact. The lifestatus/3 rule successfully joined the age/2 and status/2 relations to classify each hero as young or old. All outputs matched the expected results obtained through manual trace of the unification and backtracking process, verifying the correctness of the knowledge base and rule definitions.

Result

A Prolog knowledge base modelling Avengers heroes, villains, their powers, weapons, and ages was successfully designed and implemented. Rules for deriving allies, fight combinations, and life status were implemented and tested successfully, and all queries produced correct and complete results including multiple solutions via backtracking.

Conclusion

Through this experiment, the fundamentals of logic programming in Prolog were studied and applied by designing a knowledge base of facts and rules based on the Avengers theme. The experiment provided a clear understanding of unification, backtracking, and rule-based inference, and demonstrated how new knowledge can be derived from existing facts using simple, declarative rules rather than explicit step-by-step procedures.